In [1]:
```
import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.metrics import classification_report, confusion_matrix
from sklearn.utils.class_weight import compute_class_weight

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout, BatchNormalization
from tensorflow.keras.callbacks import EarlyStopping
from tensorflow.keras.utils import to_categorical
```

In [3]:
```
df = pd.read_csv("raw_table.csv")

# Quitar registros sin segmento
df_model = df.dropna(subset=["ATSEG"]).copy()

# Crear variables extra
df_model["TOTAL_PRESCRIPTIONS"] = (
    df_model["UC_TRX"] + df_model["ORAL_TRX"] + df_model["IL23_TRX"]
)

df_model["ENGAGEMENT_INDEX"] = (
    df_model["RTE"] + df_model["DETAILS"] + df_model["SAMPLES"]
)

df_model["ENGAGEMENT_RATIO"] = df_model["ENGAGEMENT_INDEX"] / (
    df_model["TOTAL_PRESCRIPTIONS"] + 1
)
```

In [4]:
```
features_to_agg = [
    "TOTAL_PRESCRIPTIONS",
    "ENGAGEMENT_INDEX",
    "ENGAGEMENT_RATIO",
    "UC_TRX",
    "ORAL_TRX",
    "IL23_TRX",
    "RTE",
    "SAMPLES",
    "DETAILS",
    "COPAY",
    "DIRECTMAIL",
    "SPK"
]

agg_df = df_model.groupby("NUEVO_ID").agg({
    **{col: ["mean", "max", "sum"] for col in features_to_agg},
    "ATSEG": "first"
})

agg_df.columns = ["_".join(col).strip() if col[1] else col[0] for col in agg_df.columns]
agg_df = agg_df.reset_index()
```

In [5]:
```
X = agg_df.drop(columns=["NUEVO_ID", "ATSEG_first"])
y = agg_df["ATSEG_first"]

le = LabelEncoder()
y_encoded = le.fit_transform(y)
y_cat = to_categorical(y_encoded)

X_train, X_test, y_train, y_test = train_test_split(
    X, y_cat,
    test_size=0.2,
    random_state=42,
    stratify=y_encoded
)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

In [6]:
```
class_weights = compute_class_weight(
    class_weight="balanced",
```

```
    classes=np.unique(y_encoded),
    y=y_encoded
)

class_weights = dict(enumerate(class_weights))
class_weights
```

Out[6]:
```
{0: np.float64(0.6191591216567801),
 1: np.float64(1.1843336319299294),
 2: np.float64(1.849968905472637)}
```

In [7]:
```
model = Sequential([
    Dense(128, activation="relu", input_shape=(X_train_scaled.shape[1],)),
    BatchNormalization(),
    Dropout(0.3),

    Dense(64, activation="relu"),
    BatchNormalization(),
    Dropout(0.3),

    Dense(32, activation="relu"),
    Dropout(0.2),

    Dense(3, activation="softmax")
])

model.compile(
    optimizer="adam",
    loss="categorical_crossentropy",
    metrics=["accuracy"]
)
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

In [8]:
```
early_stop = EarlyStopping(
    monitor="val_loss",
    patience=5,
    restore_best_weights=True
)

history = model.fit(
    X_train_scaled,
    y_train,
    validation_split=0.2,
    epochs=50,
    batch_size=64,
    class_weight=class_weights,
    callbacks=[early_stop],
    verbose=1
)
```

```
Epoch 1/50
119/119 ━━━━━━━━━━━━━━━━━━━━ 3s 6ms/step - accuracy: 0.4760 - loss: 1.2224 - val_accuracy: 0.5903 - val_loss: 0.953
2
Epoch 2/50
119/119 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.5246 - loss: 1.0541 - val_accuracy: 0.5856 - val_loss: 0.950
3
Epoch 3/50
119/119 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.5359 - loss: 1.0125 - val_accuracy: 0.5951 - val_loss: 0.910
0
Epoch 4/50
119/119 ━━━━━━━━━━━━━━━━━━━━ 2s 10ms/step - accuracy: 0.5467 - loss: 0.9907 - val_accuracy: 0.5961 - val_loss: 0.89
91
Epoch 5/50
119/119 ━━━━━━━━━━━━━━━━━━━━ 2s 13ms/step - accuracy: 0.5603 - loss: 0.9838 - val_accuracy: 0.5945 - val_loss: 0.89
88
Epoch 6/50
119/119 ━━━━━━━━━━━━━━━━━━━━ 1s 9ms/step - accuracy: 0.5728 - loss: 0.9587 - val_accuracy: 0.5888 - val_loss: 0.920
7
Epoch 7/50
119/119 ━━━━━━━━━━━━━━━━━━━━ 1s 9ms/step - accuracy: 0.5836 - loss: 0.9500 - val_accuracy: 0.5888 - val_loss: 0.910
7
Epoch 8/50
119/119 ━━━━━━━━━━━━━━━━━━━━ 1s 10ms/step - accuracy: 0.5796 - loss: 0.9475 - val_accuracy: 0.5898 - val_loss: 0.91
81
Epoch 9/50
119/119 ━━━━━━━━━━━━━━━━━━━━ 1s 10ms/step - accuracy: 0.5936 - loss: 0.9391 - val_accuracy: 0.5945 - val_loss: 0.90
74
Epoch 10/50
119/119 ━━━━━━━━━━━━━━━━━━━━ 1s 11ms/step - accuracy: 0.5903 - loss: 0.9345 - val_accuracy: 0.6129 - val_loss: 0.88
63
Epoch 11/50
119/119 ━━━━━━━━━━━━━━━━━━━━ 2s 9ms/step - accuracy: 0.5946 - loss: 0.9366 - val_accuracy: 0.5914 - val_loss: 0.910
6
Epoch 12/50
119/119 ━━━━━━━━━━━━━━━━━━━━ 1s 11ms/step - accuracy: 0.5925 - loss: 0.9367 - val_accuracy: 0.5993 - val_loss: 0.90
66
Epoch 13/50
119/119 ━━━━━━━━━━━━━━━━━━━━ 3s 14ms/step - accuracy: 0.6011 - loss: 0.9292 - val_accuracy: 0.6040 - val_loss: 0.88
97
Epoch 14/50
119/119 ━━━━━━━━━━━━━━━━━━━━ 1s 11ms/step - accuracy: 0.6029 - loss: 0.9285 - val_accuracy: 0.6024 - val_loss: 0.89
87
Epoch 15/50
119/119 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.6005 - loss: 0.9299 - val_accuracy: 0.6024 - val_loss: 0.909
2
```

In [9]:

```python
y_pred_prob = model.predict(X_test_scaled)
y_pred = np.argmax(y_pred_prob, axis=1)
y_true = np.argmax(y_test, axis=1)

print(classification_report(
    y_true,
    y_pred,
    target_names=le.classes_
))

print(confusion_matrix(y_true, y_pred))
```

```
75/75 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step
              precision    recall  f1-score   support

       SEG_A       0.77      0.72      0.74      1281
       SEG_B       0.50      0.52      0.51       670
       SEG_C       0.35      0.39      0.37       429

    accuracy                           0.61      2380
   macro avg       0.54      0.54      0.54      2380
weighted avg       0.62      0.61      0.61      2380

[[925 200 156]
 [171 347 152]
 [108 153 168]]
```

Modelo CNN

In [10]:
```
X_train_cnn = X_train_scaled.reshape(X_train_scaled.shape[0], X_train_scaled.shape[1], 1)
X_test_cnn = X_test_scaled.reshape(X_test_scaled.shape[0], X_test_scaled.shape[1], 1)
```

In [11]:
```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv1D, MaxPooling1D, Flatten, Dense, Dropout, BatchNormalization
from tensorflow.keras.callbacks import EarlyStopping

model_cnn = Sequential([
    Conv1D(filters=64, kernel_size=3, activation="relu", input_shape=(X_train_cnn.shape[1], 1)),
    BatchNormalization(),
    MaxPooling1D(pool_size=2),
    Dropout(0.3),

    Conv1D(filters=32, kernel_size=3, activation="relu"),
    BatchNormalization(),
    Dropout(0.3),

    Flatten(),

    Dense(64, activation="relu"),
    Dropout(0.3),

    Dense(3, activation="softmax")
])

model_cnn.compile(
    optimizer="adam",
    loss="categorical_crossentropy",
    metrics=["accuracy"]
)
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass a
n `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` objec
t as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

In [12]:
```
early_stop = EarlyStopping(
    monitor="val_loss",
    patience=6,
    restore_best_weights=True
)

history_cnn = model_cnn.fit(
    X_train_cnn,
    y_train,
    validation_split=0.2,
    epochs=60,
    batch_size=64,
    class_weight=class_weights,
    callbacks=[early_stop],
    verbose=1
)
```

```
Epoch 1/60
119/119 ━━━━━━━━━━━━━━━━━━━━ 8s 20ms/step - accuracy: 0.5089 - loss: 1.1901 - val_accuracy: 0.6303 - val_loss: 0.97
82
Epoch 2/60
119/119 ━━━━━━━━━━━━━━━━━━━━ 2s 14ms/step - accuracy: 0.5628 - loss: 1.0007 - val_accuracy: 0.6066 - val_loss: 0.92
88
Epoch 3/60
119/119 ━━━━━━━━━━━━━━━━━━━━ 2s 17ms/step - accuracy: 0.5663 - loss: 0.9819 - val_accuracy: 0.5672 - val_loss: 0.95
41
Epoch 4/60
119/119 ━━━━━━━━━━━━━━━━━━━━ 2s 20ms/step - accuracy: 0.5859 - loss: 0.9505 - val_accuracy: 0.5851 - val_loss: 0.91
05
Epoch 5/60
119/119 ━━━━━━━━━━━━━━━━━━━━ 1s 12ms/step - accuracy: 0.5877 - loss: 0.9400 - val_accuracy: 0.6176 - val_loss: 0.87
72
Epoch 6/60
119/119 ━━━━━━━━━━━━━━━━━━━━ 2s 14ms/step - accuracy: 0.5957 - loss: 0.9395 - val_accuracy: 0.5987 - val_loss: 0.88
50
Epoch 7/60
119/119 ━━━━━━━━━━━━━━━━━━━━ 2s 14ms/step - accuracy: 0.5913 - loss: 0.9286 - val_accuracy: 0.5877 - val_loss: 0.91
17
Epoch 8/60
119/119 ━━━━━━━━━━━━━━━━━━━━ 1s 12ms/step - accuracy: 0.5919 - loss: 0.9291 - val_accuracy: 0.6024 - val_loss: 0.87
47
Epoch 9/60
119/119 ━━━━━━━━━━━━━━━━━━━━ 1s 12ms/step - accuracy: 0.5991 - loss: 0.9204 - val_accuracy: 0.6124 - val_loss: 0.87
20
Epoch 10/60
119/119 ━━━━━━━━━━━━━━━━━━━━ 2s 14ms/step - accuracy: 0.6005 - loss: 0.9223 - val_accuracy: 0.5998 - val_loss: 0.89
12
Epoch 11/60
119/119 ━━━━━━━━━━━━━━━━━━━━ 2s 20ms/step - accuracy: 0.5997 - loss: 0.9218 - val_accuracy: 0.5930 - val_loss: 0.88
15
Epoch 12/60
119/119 ━━━━━━━━━━━━━━━━━━━━ 2s 16ms/step - accuracy: 0.6072 - loss: 0.9137 - val_accuracy: 0.6082 - val_loss: 0.87
34
Epoch 13/60
119/119 ━━━━━━━━━━━━━━━━━━━━ 1s 12ms/step - accuracy: 0.6068 - loss: 0.9186 - val_accuracy: 0.6035 - val_loss: 0.86
58
Epoch 14/60
119/119 ━━━━━━━━━━━━━━━━━━━━ 2s 14ms/step - accuracy: 0.6042 - loss: 0.9137 - val_accuracy: 0.5846 - val_loss: 0.88
92
Epoch 15/60
119/119 ━━━━━━━━━━━━━━━━━━━━ 1s 12ms/step - accuracy: 0.6017 - loss: 0.9138 - val_accuracy: 0.6003 - val_loss: 0.87
22
Epoch 16/60
119/119 ━━━━━━━━━━━━━━━━━━━━ 2s 13ms/step - accuracy: 0.6087 - loss: 0.9137 - val_accuracy: 0.5914 - val_loss: 0.89
28
Epoch 17/60
119/119 ━━━━━━━━━━━━━━━━━━━━ 2s 13ms/step - accuracy: 0.5999 - loss: 0.9085 - val_accuracy: 0.6056 - val_loss: 0.87
44
Epoch 18/60
119/119 ━━━━━━━━━━━━━━━━━━━━ 1s 11ms/step - accuracy: 0.6076 - loss: 0.9086 - val_accuracy: 0.5961 - val_loss: 0.87
60
Epoch 19/60
119/119 ━━━━━━━━━━━━━━━━━━━━ 3s 22ms/step - accuracy: 0.6139 - loss: 0.8989 - val_accuracy: 0.5783 - val_loss: 0.89
41
```

In [13]:

```python
y_pred_prob = model_cnn.predict(X_test_cnn)
y_pred = np.argmax(y_pred_prob, axis=1)
y_true = np.argmax(y_test, axis=1)

print(classification_report(
    y_true,
    y_pred,
    target_names=le.classes_
))

print(confusion_matrix(y_true, y_pred))
```

```
75/75 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step
              precision    recall  f1-score   support

       SEG_A       0.77      0.74      0.75      1281
       SEG_B       0.51      0.46      0.48       670
       SEG_C       0.38      0.48      0.42       429

    accuracy                           0.61      2380
   macro avg       0.55      0.56      0.55      2380
weighted avg       0.62      0.61      0.62      2380

[[948 183 150]
 [182 306 182]
 [108 116 205]]
```

fine tunning

In [16]:
```
weights_list = [
    {0: 1.0, 1: 1.2, 2: 2.0},
    {0: 1.0, 1: 1.2, 2: 2.5},
    {0: 1.0, 1: 1.3, 2: 3.0},
    {0: 1.0, 1: 1.5, 2: 3.5}
]
```

In [17]:
```
history_cnn = model_cnn.fit(
    X_train_cnn,
    y_train,
    validation_split=0.2,
    epochs=60,
    batch_size=64,
    class_weight={0: 1.0, 1: 1.3, 2: 3.0},
    callbacks=[early_stop],
    verbose=1
)
```

```
Epoch 1/60
119/119 ━━━━━━━━━━━━━━━━━━━━ 2s 14ms/step - accuracy: 0.5861 - loss: 1.3068 - val_accuracy: 0.5882 - val_loss: 0.87
14
Epoch 2/60
119/119 ━━━━━━━━━━━━━━━━━━━━ 2s 13ms/step - accuracy: 0.5883 - loss: 1.3041 - val_accuracy: 0.5777 - val_loss: 0.89
06
Epoch 3/60
119/119 ━━━━━━━━━━━━━━━━━━━━ 2s 20ms/step - accuracy: 0.5982 - loss: 1.2900 - val_accuracy: 0.5798 - val_loss: 0.88
46
Epoch 4/60
119/119 ━━━━━━━━━━━━━━━━━━━━ 2s 16ms/step - accuracy: 0.5869 - loss: 1.2814 - val_accuracy: 0.5903 - val_loss: 0.87
10
Epoch 5/60
119/119 ━━━━━━━━━━━━━━━━━━━━ 2s 12ms/step - accuracy: 0.5958 - loss: 1.2836 - val_accuracy: 0.5987 - val_loss: 0.86
37
Epoch 6/60
119/119 ━━━━━━━━━━━━━━━━━━━━ 2s 13ms/step - accuracy: 0.5936 - loss: 1.2879 - val_accuracy: 0.5872 - val_loss: 0.87
78
Epoch 7/60
119/119 ━━━━━━━━━━━━━━━━━━━━ 4s 25ms/step - accuracy: 0.5925 - loss: 1.2876 - val_accuracy: 0.5940 - val_loss: 0.87
08
Epoch 8/60
119/119 ━━━━━━━━━━━━━━━━━━━━ 5s 26ms/step - accuracy: 0.5989 - loss: 1.2789 - val_accuracy: 0.5704 - val_loss: 0.89
74
Epoch 9/60
119/119 ━━━━━━━━━━━━━━━━━━━━ 2s 13ms/step - accuracy: 0.6004 - loss: 1.2795 - val_accuracy: 0.5683 - val_loss: 0.90
62
Epoch 10/60
119/119 ━━━━━━━━━━━━━━━━━━━━ 1s 12ms/step - accuracy: 0.5915 - loss: 1.2745 - val_accuracy: 0.5830 - val_loss: 0.86
87
Epoch 11/60
119/119 ━━━━━━━━━━━━━━━━━━━━ 2s 13ms/step - accuracy: 0.6008 - loss: 1.2662 - val_accuracy: 0.5819 - val_loss: 0.86
90
```

In [18]:
```
model_cnn = Sequential([
    Conv1D(128, kernel_size=5, activation="relu", input_shape=(X_train_cnn.shape[1], 1)),
    BatchNormalization(),
    MaxPooling1D(pool_size=2),
    Dropout(0.35),

    Conv1D(64, kernel_size=3, activation="relu"),
```

```
    BatchNormalization(),
    Dropout(0.30),

    Conv1D(32, kernel_size=3, activation="relu"),
    BatchNormalization(),
    Dropout(0.25),

    Flatten(),

    Dense(64, activation="relu"),
    Dropout(0.30),

    Dense(3, activation="softmax")
])

model_cnn.compile(
    optimizer="adam",
    loss="categorical_crossentropy",
    metrics=["accuracy"]
)
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass a
n `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` objec
t as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

In [19]:
```
df_model["IL23_RATIO"] = df_model["IL23_TRX"] / (df_model["TOTAL_PRESCRIPTIONS"] + 1)
df_model["ORAL_RATIO"] = df_model["ORAL_TRX"] / (df_model["TOTAL_PRESCRIPTIONS"] + 1)
df_model["UC_RATIO"] = df_model["UC_TRX"] / (df_model["TOTAL_PRESCRIPTIONS"] + 1)

df_model["RTE_RATIO"] = df_model["RTE"] / (df_model["ENGAGEMENT_INDEX"] + 1)
df_model["DETAILS_RATIO"] = df_model["DETAILS"] / (df_model["ENGAGEMENT_INDEX"] + 1)
df_model["SAMPLES_RATIO"] = df_model["SAMPLES"] / (df_model["ENGAGEMENT_INDEX"] + 1)

df_model["RX_PER_ENGAGEMENT"] = df_model["TOTAL_PRESCRIPTIONS"] / (df_model["ENGAGEMENT_INDEX"] + 1)
```

In [20]:
```
features_to_agg = [
    "TOTAL_PRESCRIPTIONS",
    "ENGAGEMENT_INDEX",
    "ENGAGEMENT_RATIO",
    "RX_PER_ENGAGEMENT",

    "UC_TRX",
    "ORAL_TRX",
    "IL23_TRX",

    "IL23_RATIO",
    "ORAL_RATIO",
    "UC_RATIO",

    "RTE",
    "SAMPLES",
    "DETAILS",

    "RTE_RATIO",
    "DETAILS_RATIO",
    "SAMPLES_RATIO",

    "COPAY",
    "DIRECTMAIL",
    "SPK"
]
```

In [21]:
```
agg_df = df_model.groupby("NUEVO_ID").agg({
    **{col: ["mean", "max", "sum", "std"] for col in features_to_agg},
    "ATSEG": "first"
})

agg_df.columns = ["_".join(col).strip() if col[1] else col[0] for col in agg_df.columns]
agg_df = agg_df.reset_index()

agg_df = agg_df.fillna(0)
```

In [23]:
```
from sklearn.metrics import f1_score, accuracy_score, recall_score
```

In [24]:
```
from sklearn.metrics import f1_score, accuracy_score

results = []

results.append({
    "model": "CNN",
    "accuracy": accuracy_score(y_true, y_pred),
    "macro_f1": f1_score(y_true, y_pred, average="macro"),
    "weighted_f1": f1_score(y_true, y_pred, average="weighted"),
    "seg_c_f1": f1_score(y_true, y_pred, average=None)[2],
    "seg_c_recall": recall_score(y_true, y_pred, average=None)[2]
})
```

In [25]:
```
from sklearn.metrics import f1_score, accuracy_score, recall_score
```

In [26]:
```
print(le.classes_)
```

```
['SEG_A' 'SEG_B' 'SEG_C']
```

In [27]:
```
print(results)
```

```
[{'model': 'CNN', 'accuracy': 0.6130252100840337, 'macro_f1': 0.5523700921992168, 'weighted_f1': 0.616749309869450
6, 'seg_c_f1': np.float64(0.4244306418219462), 'seg_c_recall': np.float64(0.47785547785547783)}]
```

In [28]:
```
print(classification_report(y_true, y_pred))
```

```
              precision    recall  f1-score   support

           0       0.77      0.74      0.75      1281
           1       0.51      0.46      0.48       670
           2       0.38      0.48      0.42       429

    accuracy                           0.61      2380
   macro avg       0.55      0.56      0.55      2380
weighted avg       0.62      0.61      0.62      2380
```

Otro autotunning

In [30]:
```
!pip install keras-tuner
```

```
Collecting keras-tuner
  Downloading keras_tuner-1.4.8-py3-none-any.whl.metadata (5.6 kB)
Requirement already satisfied: keras in /usr/local/lib/python3.12/dist-packages (from keras-tuner) (3.13.2)
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packages (from keras-tuner) (26.1)
Requirement already satisfied: requests in /usr/local/lib/python3.12/dist-packages (from keras-tuner) (2.32.4)
Collecting kt-legacy (from keras-tuner)
  Downloading kt_legacy-1.0.5-py3-none-any.whl.metadata (221 bytes)
Requirement already satisfied: grpcio in /usr/local/lib/python3.12/dist-packages (from keras-tuner) (1.80.0)
Requirement already satisfied: protobuf in /usr/local/lib/python3.12/dist-packages (from keras-tuner) (5.29.6)
Requirement already satisfied: typing-extensions~=4.12 in /usr/local/lib/python3.12/dist-packages (from grpcio->ker
as-tuner) (4.15.0)
Requirement already satisfied: absl-py in /usr/local/lib/python3.12/dist-packages (from keras->keras-tuner) (1.4.0)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (from keras->keras-tuner) (2.0.2)
Requirement already satisfied: rich in /usr/local/lib/python3.12/dist-packages (from keras->keras-tuner) (13.9.4)
Requirement already satisfied: namex in /usr/local/lib/python3.12/dist-packages (from keras->keras-tuner) (0.1.0)
Requirement already satisfied: h5py in /usr/local/lib/python3.12/dist-packages (from keras->keras-tuner) (3.16.0)
Requirement already satisfied: optree in /usr/local/lib/python3.12/dist-packages (from keras->keras-tuner) (0.19.0)
Requirement already satisfied: ml-dtypes in /usr/local/lib/python3.12/dist-packages (from keras->keras-tuner) (0.5.
4)
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests->
keras-tuner) (3.4.7)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests->keras-tuner)
(3.13)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests->keras-
tuner) (2.5.0)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests->keras-
tuner) (2026.4.22)
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3.12/dist-packages (from rich->keras->
keras-tuner) (4.0.0)
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/python3.12/dist-packages (from rich->keras
->keras-tuner) (2.20.0)
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.12/dist-packages (from markdown-it-py>=2.2.0->r
ich->keras->keras-tuner) (0.1.2)
Downloading keras_tuner-1.4.8-py3-none-any.whl (129 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 129.4/129.4 kB 2.2 MB/s eta 0:00:00
Downloading kt_legacy-1.0.5-py3-none-any.whl (9.6 kB)
Installing collected packages: kt-legacy, keras-tuner
Successfully installed keras-tuner-1.4.8 kt-legacy-1.0.5
```

In [31]:

```python
import keras_tuner as kt
import tensorflow as tf

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout, BatchNormalization
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.callbacks import EarlyStopping

from sklearn.metrics import classification_report, confusion_matrix, f1_score, accuracy_score, recall_score
```

In [32]:

```python
def build_model(hp):
    model = Sequential()

    model.add(Dense(
        units=hp.Int("units_1", min_value=64, max_value=256, step=64),
        activation="relu",
        input_shape=(X_train_scaled.shape[1],)
    ))
    model.add(BatchNormalization())
    model.add(Dropout(hp.Float("dropout_1", min_value=0.2, max_value=0.5, step=0.1)))

    for i in range(hp.Int("num_layers", 1, 3)):
        model.add(Dense(
            units=hp.Int(f"units_{i+2}", min_value=32, max_value=128, step=32),
            activation="relu"
        ))
        model.add(BatchNormalization())
        model.add(Dropout(hp.Float(f"dropout_{i+2}", min_value=0.2, max_value=0.5, step=0.1)))

    model.add(Dense(3, activation="softmax"))

    learning_rate = hp.Choice("learning_rate", values=[0.001, 0.0005, 0.0001])

    model.compile(
        optimizer=Adam(learning_rate=learning_rate),
        loss="categorical_crossentropy",
        metrics=["accuracy"]
    )
```

```
    return model
```

In [33]:
```
tuner = kt.RandomSearch(
    build_model,
    objective="val_accuracy",
    max_trials=15,
    executions_per_trial=1,
    directory="tuning_results",
    project_name="segment_nn_tuning"
)
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

In [34]:
```
early_stop = EarlyStopping(
    monitor="val_loss",
    patience=5,
    restore_best_weights=True
)

tuner.search(
    X_train_scaled,
    y_train,
    validation_split=0.2,
    epochs=50,
    batch_size=64,
    class_weight={0: 1.0, 1: 1.3, 2: 3.0},
    callbacks=[early_stop],
    verbose=1
)
```

```
Trial 15 Complete [00h 00m 15s]
val_accuracy: 0.5835084319114685

Best val_accuracy So Far: 0.6171218752861023
Total elapsed time: 00h 06m 28s
```

In [35]:
```
best_model = tuner.get_best_models(num_models=1)[0]
best_hps = tuner.get_best_hyperparameters(num_trials=1)[0]

print("Best hyperparameters:")
print(best_hps.values)
```

```
Best hyperparameters:
{'units_1': 256, 'dropout_1': 0.4, 'num_layers': 1, 'units_2': 32, 'dropout_2': 0.30000000000000004, 'learning_rate': 0.001, 'units_3': 32, 'dropout_3': 0.4, 'units_4': 96, 'dropout_4': 0.2}
/usr/local/lib/python3.12/dist-packages/keras/src/saving/saving_lib.py:797: UserWarning: Skipping variable loading for optimizer 'adam', because it has 2 variables whereas the saved optimizer has 22 variables.
  saveable.load_own_variables(weights_store.get(inner_path))
```

In [36]:
```
y_pred_prob = best_model.predict(X_test_scaled)
y_pred = y_pred_prob.argmax(axis=1)
y_true = y_test.argmax(axis=1)

print(classification_report(
    y_true,
    y_pred,
    target_names=le.classes_
))

print(confusion_matrix(y_true, y_pred))
```

```
75/75 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step
              precision    recall  f1-score   support

       SEG_A       0.68      0.89      0.77      1281
       SEG_B       0.51      0.29      0.37       670
       SEG_C       0.41      0.31      0.35       429

    accuracy                           0.62      2380
   macro avg       0.53      0.50      0.50      2380
weighted avg       0.58      0.62      0.58      2380

[[1136   97   48]
 [ 328  197  145]
 [ 207   89  133]]
```

In [37]:

```python
f1_per_class = f1_score(y_true, y_pred, average=None)
recall_per_class = recall_score(y_true, y_pred, average=None)

results.append({
    "model": "Tuned_NN",
    "accuracy": float(accuracy_score(y_true, y_pred)),
    "macro_f1": float(f1_score(y_true, y_pred, average="macro")),
    "weighted_f1": float(f1_score(y_true, y_pred, average="weighted")),
    "seg_c_f1": float(f1_per_class[2]),
    "seg_c_recall": float(recall_per_class[2])
})

pd.DataFrame(results).round(3)
```

Out[37]:

	model	accuracy	macro_f1	weighted_f1	seg_c_f1	seg_c_recall
0	CNN	0.613	0.552	0.617	0.424	0.478
1	Tuned_NN	0.616	0.499	0.583	0.352	0.310